Process Agent — AI 时代的第二个产品物种

> **文档目的**：为团队建立"我们到底在做什么"的共同语言。本文档分两部分：
>
> - 上半部分（§1-§5）讲清 Task 型 vs Process 型的分野，以及为什么 Process 型是新的产品机会
> - 下半部分（§6-§10）讲清楚作为 Process Agent 公司，我们底层要构建一个什么样的通用技术架构与能力
>
> **文档版本**：v1.0 **目标读者**：团队全员（战略 + 产品 + 工程通读）

目录

第一部分 战略与品类

1. 背景：当下 AI 产品的隐藏假设
2. 新的物种分类：Task Agent vs Process Agent
3. Process Agent 的本质特征
4. 为什么 Process Agent 是新的机会
5. Process Agent 的市场地图

第二部分 架构与能力

6. 核心抽象：Process Agent Engine
7. 必备能力 1: Behavior Stream（行为流）
8. 必备能力 2: User State Engine（状态引擎）
9. 必备能力 3: Daily Ritual Framework（每日仪式框架）
10. 必备能力 4: Companion Engine（陪伴引擎）
11. 必备能力 5: Proactive Nudge Engine（主动推送引擎）
12. 架构总览与领域适配模式
13. 实施路径与里程碑

第一部分：战略与品类

1. 背景：当下 AI 产品的隐藏假设

1.1 当下主流 AI 产品的形态

观察 2024-2026 年最具代表性的 AI 产品 —— ChatGPT、Claude、Gemini、Cursor、Codex、Manus、OpenClaw、Devin —— 它们在产品形态上有一个共同特征：

用户的需求被理解为"在一个 session 内可以闭环的 task"。

具体说：

- 用户进入一个对话 / 工作环境
- 描述需求或目标
- AI 调用工具、推理、产出结果
- session 结束，问题解决

这些产品的工程优化方向也都围绕这个假设：

- **更长的 context window**（让单 session 能处理更多信息）
- **更多的工具集成**（让单 session 能完成更多类型的事情）
- **更强的推理深度**（让单 session 内的决策更靠谱）
- **更精细的 agent 编排**（让单 session 内多步骤执行更可靠）

1.2 这个假设的隐藏限制

"Task 型"假设暗含一个前提：**用户的需求是离散的、独立的、可以在有限时间内解决的**。

这对一大类需求是成立的：

- 写一段代码 ✓
- 解决一个 bug ✓
- 写一封邮件 ✓
- 做一份研究 ✓
- 订一张机票 ✓
- 总结一篇论文 ✓

但对另一大类需求是不成立的：

- 学好一门 AP 课程 ✗（持续 9 个月）
- 改善焦虑情绪 ✗（持续多年）
- 减脂 20 磅并保持 ✗（持续 6 个月+）

- 学会一门外语到能日常交流 ✗（持续 2-3 年）
- 培养一个新习惯 ✗（持续 3-12 个月）
- 成为更好的伴侣 / 父母 ✗（持续一辈子）

这两类需求在产品形态上的差异是根本性的，不能用同一种产品架构服务。

1.3 关键洞察

当下所有主流 AI 产品都在解决第一类需求。

这意味着第二类需求形成了一个完整的、未被认领的产品空间。这个空间就是我们说的 Process Agent。

---

2. 新的物种分类：Task Agent vs Process Agent

2.1 两种物种的本质区别

维度	Task Agent	Process Agent
用户需求形态	离散、独立的目标	连续、累积的过程
时间尺度	分钟到小时	周到年
session 关系	每个 session 独立	跨 session 持续连贯
AI 角色	工具 / 助手	伙伴 / 教练
触发机制	用户主动请求	AI 主动 + 用户响应
价值衡量	单次任务完成质量	长期目标达成度
核心数据资产	session 内的 context	跨 session 的用户状态模型
技术优化方向	推理深度、工具集成	用户状态建模、主动时机判断
护城河来源	模型能力 + 工具广度	长期行为数据 + 状态精度
典型产品	ChatGPT, Claude, Cursor	(新品类)
类比	私人助理	私人教练 / 私人医生

2.2 一个直观的类比

想象你雇佣两个人：

- 一个是**外包顾问**：你有具体问题就找他，他帮你解决，按次付费
- 一个是**私人教练**：他记得你的健康历史，每周来训练你，主动调整你的计划

两者都"提供价值"，但产品形态完全不同：

- 顾问优化的是"被调用时回答问题的能力"
- 教练优化的是"对你长期了解 + 主动推动你成长的能力"

ChatGPT 是顾问。Process Agent 是教练。

这不是优劣之分，是产品物种之分。它们服务的是用户生活中两类完全不同的需求。

2.3 为什么这是"两个物种"，不是"一个连续光谱"

可能有人会说：能不能让 ChatGPT 也变得"长期记忆 + 主动陪伴"，不就成 Process Agent 了？

答案是不能。原因有三个：

原因 1: 产品形态根本不同

ChatGPT 的入口是"空白对话框"。Process Agent 的入口必须是"今天的任务包 / 今日仪式"。这两种入口塑造了完全不同的用户心智 —— 一个是"我需要时来用"，一个是"它每天等我来"。

原因 2: 数据架构根本不同

Task Agent 优化的是"session 内的 context 利用"。Process Agent 优化的是"跨 session 的 user state representation"。前者是 transient state，后者是 persistent state。两者的数据模型设计、生命周期管理、检索策略全部不同。

原因 3: 商业模式根本不同

Task Agent 是 usage-based（用得多付得多）。Process Agent 是 subscription-based + retention-driven（按月订阅，靠长期留存）。这意味着获客、定价、产品决策的所有 key metric 都不同。

结论：

Task Agent 和 Process Agent 是两个独立的产品物种，不会通过简单演化互相变成对方。新机会必须有新公司专门做。

2.4 这两个物种的市场结构

Task Agent 市场：已经被 OpenAI / Anthropic / Google 等基础模型公司占据。创业公司在这里很难突破，因为这个品类的护城河是"基础模型能力 + 工具集成广度"，而这两者都是大厂的天然优势。

Process Agent 市场：基本是空的。现有产品要么是"前 AI 时代的伪 AI"（Duolingo, Headspace, Strava 等），要么是某些 task agent 试图加入"记忆"功能的浅层尝试。**没有一家公司在这个品类上 all-in**。

这个空缺不是因为没人想到，是因为做这个品类需要的能力组合（深领域知识 + AI Native 设计 + 长期数据飞轮思维）不在大厂的舒适区，也不在传统创业公司的想象里。

---

3. Process Agent 的本质特征

任何一个真正的 Process Agent 产品都必须同时具备以下五个特征。缺一个就退化成其他类型的产品。

3.1 特征 1: 跨 session 全局记忆

这是最根本的特征。Process Agent 必须有一个**持续累积的、能准确描述用户当前状态的记忆系统**。

不是 ChatGPT Memory 那种浅层 profile（"用户喜欢简洁的回答"），而是**对用户在某个生活领域的深度状态建模**：

- 学习领域：用户在每个 concept 的掌握度、哪里卡过、考试日期
- 心理领域：用户的情绪基线、易触发因素、有效的应对策略
- 健身领域：用户的体能参数、训练历史、伤病史、饮食偏好

这层记忆是 Process Agent 一切其他能力的基础。**它不是"feature"，是"infrastructure"**。

3.2 特征 2: Daily Ritual（每日仪式）

Process Agent 必须有一个**用户每天会主动打开的"固定动作"**。这是产品形成习惯回路的关键。

这个 daily ritual 的特征：

- **时长固定且可预测**（10-20 分钟，不是任意长）
- **内容个性化**（基于全局记忆每日重新生成）
- **完成后有反馈**（streak、能力可视化、角色互动）
- **完成是有限的**（"今天就这么多"，不是无限刷）

为什么这一点关键？因为 Process 型需求本质上是反人性的（学习、健身、心理建设都需要克服惰性）。**没有 daily ritual，用户没有"每天打开"的理由，长期记忆也就没有数据来源**。

Daily ritual 是数据飞轮的"轮轴"。

3.3 特征 3: 陪伴角色（Companion）

Process Agent 必须有一个**有人格的角色**，不是一个无名的 AI。

为什么？因为 Process 型需求的成功，**很大程度上取决于情感连接**。学习一门难课、改善焦虑、保持健身 —— 这些目标的达成不是靠工具，靠的是"有人陪我"的感觉。

这个角色需要：

- 一致的视觉形象
- 一致的 voice 和 personality
- 会成长、有情绪、能和用户建立关系
- 是产品的"灵魂载体"，而不是吉祥物

ChatGPT 永远不会有"角色"。它的设计哲学就是中性的工具。Process Agent 的设计哲学必须包含"有灵魂"。

3.4 特征 4: 主动行为（Proactivity）

这一点是 Process Agent 区别于"加了记忆的 ChatGPT"的根本。

Task Agent 是**被动的**：用户提问，它回答。 Process Agent 必须是**主动的**：在用户需要时（不是请求时）主动出现。

具体形态：

- 主动推送（"Maya 你的 Loop 准备好了"）
- 主动追问（"上次那道题三天后你还会做吗，我考考你？"）
- 主动调整计划（"看你这周状态不好，明天减半"）
- 主动连接（"你昨天学的概念和今天有关，让我提一下"）

这些主动行为必须基于全局记忆做出**有价值、有时机感、不打扰**的判断。这是 Process Agent 工程上最难的部分，也是最不可替代的部分。

3.5 特征 5: 可衡量的长期目标

Process Agent 必须服务于一个**用户和系统都能衡量的长期目标**。

学习：考分提升、知识掌握度 心理：情绪基线改善、焦虑频率下降 健身：体重 / 体脂 / 力量 / 耐力 的具体指标 习惯：行为坚持的天数、行为质量的变化

如果没有可衡量的长期目标，Process Agent 退化成"陪聊机器人"，无法证明价值，也无法做产品决策（不知道哪些功能真的有效）。

这一点对内容和产品的设计要求非常高 —— **每个产品决策都要回答"这是不是真的帮助用户向目标推进？"**

3.6 五个特征的相互依赖

这五个特征不是独立的 feature list，是相互依赖的有机系统：

```
长期目标 (3.5)
    ↓ 决定衡量什么
全局记忆 (3.1)
    ↓ 描述用户当前状态
每日仪式 (3.2)
    ↓ 持续产生新数据 + 提供产品形态
陪伴角色 (3.4)
    ↓ 让仪式有情感意义
主动行为 (3.5)
    ↓ 让记忆产生价值，闭环回到长期目标
```

去掉任何一个，整个系统都不工作：

- 没有全局记忆 → 无法个性化 → 沦为通用工具
- 没有每日仪式 → 用户不来 → 数据飞轮停转
- 没有陪伴角色 → 用户没情感连接 → 留存崩盘
- 没有主动行为 → 退化为被动工具 → 跟 ChatGPT 同质化
- 没有长期目标 → 无法衡量价值 → 商业模型不成立
-

---

4. 为什么 Process Agent 是新的机会

4.1 这个机会为什么是"新的"

Process 型需求（学习、心理、健身、成长）一直存在。但作为 AI Native 产品形态出现的机会窗口，**直到 2024-2026 年才真正打开**。原因有三个：

LLM 能力到位：跨 session 记忆 + 长 context 理解 + 主动决策，这三项能力在 GPT-4 / Claude 3.5 这一代之前是不可能的。 **单位经济模型可行**：LLM 调用成本下降到了月费 $10-25 能覆盖的水平。三年

前每用户每月成本会是 $100+，做不出消费级产品。 **用户心智成熟**：经历了 2 年的 ChatGPT 教育，用户已经接受"和 AI 长期互动"这件事，不需要从零教育。

这三个条件同时满足只在最近一两年发生。**这是新公司的窗口期**。

4.2 为什么大厂不会做这个

OpenAI / Anthropic / Google 不会成为 Process Agent 公司，理由是结构性的：

理由 1: 产品形态冲突

Task Agent 的核心是"通用接口 + 用户主导"。Process Agent 的核心是"垂直深度 + 系统主导"。同一个产品很难同时是这两种。强行兼容会两边都做不好。

理由 2: 数据飞轮方向不同

Task Agent 的数据飞轮是"通过亿万用户的反馈优化通用模型"。Process Agent 的数据飞轮是"通过单个用户的长期数据优化对该用户的服务"。前者是横向 broad，后者是纵向 deep。两者的工程优化方向相反。

理由 3: 商业激励不同

OpenAI 的成长目标是"占领所有人的所有 AI 使用"。这个目标和"专注服务某类用户某类需求"在战略层面冲突。即使他们想做某个 Process Agent vertical，董事会也会让他们回到平台战略上。

理由 4: 组织能力不匹配

做 Process Agent 需要深领域专家（教育、心理、健身等）+ 长期产品迭代能力 + 用户研究文化。这些都是"消费产品公司"的能力，不是"基础模型公司"的能力。OpenAI 的工程文化做不出 Duolingo / Headspace 这种产品。

结论：Process Agent 这个品类的胜出者一定是新公司。这是创业者的真正机会。

4.3 Process Agent 的护城河

在这个新品类里建立长期防守优势的来源:

护城河 1: 用户行为数据的累积

Process Agent 用户每天产生数百个数据点（学习行为、情绪记录、运动数据等）。一年累积下来，每个用户的数据质量是惊人的。**这些数据无法被新进入者模仿**，因为它本质上是时间的函数。

护城河 2: 用户与角色的情感关系

陪伴角色（Otter / 类似的角色）一旦在用户心智里建立位置，迁移成本极高。这跟用户对某个朋友的依赖类似 —— 不是另一个产品提供同样的服务就能取代的。

护城河 3: 内容运营的 know-how

什么内容能让用户每天打开？什么节奏不会让用户疲倦？什么 nudge 时机最有效？这些都是要通过几十万用户实战才能积累的产品 sense。后来者要追上需要的不是技术，是时间。

护城河 4: 跨垂直的引擎复用

如果一家公司在多个 vertical（学习 + 心理 + 健身）建立 Process Agent 产品，**底层引擎是共享的**。这创造了横向的杠杆 —— 后来者要从零开始，先发者已经摊销了 70% 的工程成本。

这四层护城河叠加，让 Process Agent 公司的长期价值非常高。

---

5. Process Agent 的市场地图

5.1 哪些垂直适合 Process Agent

不是所有需求都能做成 Process Agent。判断一个垂直是否适合，看三个条件：

1. **需求是否长期累积**（vs 一次性解决）
2. **是否能定义可衡量的目标**（vs 模糊的"过得更好"）
3. **是否值得用户每天投入 10-20 分钟**（vs 一周用一次）

按这三个条件评估常见的需求领域：

垂直	长期累积	可衡量目标	每日值得	适合度
学习 / 备考	✓✓✓	✓✓✓（考分）	✓✓✓	极适合
语言学习	✓✓✓	✓✓（流利度）	✓✓✓	极适合
心理健康	✓✓✓	✓（情绪基线）	✓✓	适合
健身 / 减脂	✓✓✓	✓✓✓（体重 / 力量）	✓✓	极适合
营养管理	✓✓	✓✓	✓	中等
习惯养成	✓✓✓	✓✓	✓✓	适合
关系经营	✓✓✓	✗	✗	不适合
财富管理	✓✓	✓✓	✗	不适合
写作 / 创作技能	✓✓	✓	✓	中等
育儿陪伴	✓✓✓	✓	✓✓	适合

最适合的垂直集中在：**学习、健身、心理、习惯**这四个领域。它们的共同特征是有清晰的可衡量目标 + 用户能感受到每日小步进展。

5.2 进入顺序的建议

如果一家公司要在多个 vertical 建立 Process Agent 产品矩阵，**进入顺序很关键**。建议:

第 1 个 vertical：选最适合积累技术的领域（结构化最强 + 用户付费意愿最高）。学习 / 备考 是最佳选择 —— 高度结构化、目标清晰、付费习惯成熟。

第 2 个 vertical：选与第 1 个有用户重合 + 引擎复用最高的领域。从学习扩展到"职业技能学习"或"语言学习"是自然延伸。

第 3 个 vertical：开始进入"非学习类"的 process 领域。这一步是验证引擎真的领域无关 —— 心理健康或健身都是好选择。

第 4 个之后：基于前 3 个验证的 backbone，可以系统性扩展。

关键判断：每一步扩展之前都要回答 "我们的引擎里多少能直接复用？"。如果低于 70%，说明引擎抽象做得不够好，要先回去补。

5.3 PrepGo 在这张地图里的位置

PrepGo（AP 备考产品）位于"学习"这个最佳起点垂直的最结构化的子领域。这是一个非常理想的"训练场"位置：

- 用户付费意愿强（验证经济模型）
- 领域结构紧（KG 投资 ROI 高）
- 目标极清晰（AP 考分）
- 用户群体集中（高中生 + 家长）
- 有相邻扩展空间（其他备考、大学课程、终身学习）

用 PrepGo 打磨 Process Agent 引擎，是从这个市场地图角度看最聪明的起手。

第二部分：架构与能力

6. 核心抽象：Process Agent Engine

6.1 引擎的定位

Process Agent Engine 是一个**领域无关的产品技术底座**。它不是一个完整产品，是构建任何 Process Agent 产品的"操作系统层"。

把它和具体的 vertical 产品的关系画清楚：

```
代码块
[Process Agent Engine]    ← 领域无关的底座
        ↑ 注入 schema / prompts / characters
  [Vertical 1: PrepGo (AP 备考)]
  [Vertical 2: 心理健康 (未来)]
  [Vertical 3: 健身 (未来)]
  [Vertical N: ...]
```

每个 vertical 产品 = 引擎 + 领域适配层。理想情况下，新 vertical 的工程量应该是 30%（领域适配），不是 100%（重新做产品）。

6.2 引擎的核心组件

一个完整的 Process Agent Engine 必须包含五个核心组件，对应第一部分讲的五个本质特征:

组件	对应特征	职责
Behavior Stream	数据采集	统一接收用户在产品里的所有行为，作为引擎的"输入端
User State Engine	全局记忆 (3.1)	维护用户在该 vertical 的长期状态模型
Daily Ritual Framework	每日仪式 (3.2)	配置和调度每日内容包，形成习惯回路
Companion Engine	陪伴角色 (3.3)	角色的人格、voice、互动机制
Proactive Nudge Engine	主动行为 (3.4)	决定何时主动出现 + 出现时说什么

Long-term goal (3.5) 不是组件，是这五个组件**共同服务的目标**。每个组件的设计都要回答"这是不是真的帮助用户向目标推进"。

6.3 引擎的设计原则

构建这个引擎时必须坚守的几条原则:

原则 1: Schema-Replaceable 所有组件不应该 hardcode 具体领域概念。学习领域里的 "topic" 在心理领域可能是 "thought pattern"，在健身领域可能是 "movement pattern"。用通用术语（schema_node, mastery_target）。

原则 2: LLM-as-default, Schema-as-optimization 默认用 LLM 做推理，schema 是优化层。这让引擎能从一开始就服务长尾领域，不需要等到 schema 准备好。

原则 3: 数据流向单向 所有组件读自 Behavior Stream，写回 User State Engine。Daily Ritual / Companion / Nudge 是 User State 的消费者。这样数据一致性容易保证。

原则 4: 领域适配通过配置而非代码 新 vertical 上线时，应该是"换配置"而不是"改代码"。配置包括：schema 定义、prompt 模板、角色定义、ritual 形态。

原则 5: 假设 LLM 会越来越便宜 今天看起来昂贵的能力（每天给每个用户做长 context 推理）一年后会便宜 50%。架构要为这个曲线设计，不要被今天的成本约束未来的能力上限。

下面五章详细讲每个组件的设计。

---

7. 必备能力 1: Behavior Stream（行为流）

7.1 它是什么

Behavior Stream 是 Process Agent Engine 的**唯一输入端**。

用户在产品里的每一个动作（拍照、答题、阅读、跳过、收藏、退出、长按、滑动到底等等）都进入这个统一的事件流。其他所有组件从这个流里读取数据，进行下游处理。

为什么必须统一？因为 Process Agent 的核心价值来自对用户行为的全面理解。如果不同动作进入不同的存储/管道，做"全局推理"时要 join 多个数据源，复杂度爆炸。**统一的事件流让"理解用户"这件事在数据层就是 O(1) 复杂度的查询**。

7.2 数据模型

一个最小够用的 Behavior Stream 数据模型:

代码块

```
CREATE TABLE behavior_events (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL,
    vertical_id TEXT NOT NULL,          -- 'prepgo' | 'mental_health' | ...

    event_type TEXT NOT NULL,           -- 'quiz_answered' | 'content_consumed' | 'photo_taken' | ...
    event_subtype TEXT,                 -- 'correct' | 'wrong' | 'skipped'

    payload JSONB NOT NULL,             -- 事件详细数据
    schema_targets TEXT[],              -- 触及的 schema 节点（topics / thought_patterns）

    timestamp TIMESTAMP NOT NULL,
    session_id UUID,                    -- 可选，用于 session 级别分析

    -- 衍生字段（可选，用于查询优化）
    derived_outcome SMALLINT,           -- -1 / 0 / +1
    derived_intensity FLOAT             -- 行为的强度信号
);
```

```
19
20  CREATE INDEX idx_events_user_time ON behavior_events(user_id, timestamp DESC);
21  CREATE INDEX idx_events_user_targets ON behavior_events(user_id,
    schema_targets);
22  CREATE INDEX idx_events_vertical_type ON behavior_events(vertical_id,
    event_type);
```

注意几个设计决策:

event_type 是开放枚举：每个 vertical 可以定义自己的事件类型。学习的 'quiz_answered' 在心理领域可能是 'mood_logged'。

payload 是 JSONB：允许每种 event_type 有完全不同的 schema，不强制统一。

schema_targets 是数组：一个事件可能触及多个 schema 节点（一道题考多个 topics）。

derived_outcome / intensity 是预计算的：避免每次查询都要从 payload 里解析。

7.3 事件类型分类

Process Agent 的事件大致分四类。每个 vertical 都有这四类，但具体形态不同:

类 A - 主动学习/练习事件：用户主动消化产品内容

- 学习: 完成 quiz 题、读 study guide、做模考
- 心理: 完成 daily check-in、做练习
- 健身: 完成 workout、记录饮食

类 B - 自由生成事件：用户主动产生内容

- 学习: 拍照解题、上传课堂笔记、写笔记
- 心理: 写日记、记录情绪、记录梦境
- 健身: 上传训练照、记录身体感受

类 C - 被动消费事件：用户接收内容但不互动

- 学习: 听播客、看 study guide 但不做题
- 心理: 听冥想引导、读文章
- 健身: 看示范视频

类 D - 元事件：跟内容无关的产品使用行为

- 打开 app、关闭 app、点击通知、跳过推送、修改设置

类 D 经常被忽视但极其重要 —— 它告诉你**用户的"意愿状态"**（是否在主动靠近产品 vs 在抵触）。

7.4 工程要点

异步写入：事件写入是高频但非关键路径。用消息队列（Kafka / Redis Streams）异步写到 DB，避免阻塞用户操作。

冷热分离：最近 30 天的事件放热存储（Postgres），历史事件迁移到冷存储（S3 + Athena 查询）。Process Agent 的查询大多关心最近行为。

Privacy 隔离：事件 payload 可能包含敏感信息（学生写的反思、心理用户的情绪）。要从 schema 设计就考虑加密 / 隔离 / 用户删除权。

Cross-vertical 隔离：同一个用户在多个 vertical 的事件应该可以互通（如果用户授权），但默认是隔离的。这是为未来 multi-vertical 用户准备的。

---

8. 必备能力 2: User State Engine（状态引擎）

8.1 它是什么

User State Engine 维护用户在某个 vertical 的**长期状态模型**。它是 Process Agent 一切个性化能力的基础。

具体说，它要回答这些问题:

- 学习场景: 这个用户在每个 topic 上掌握度多少？哪些是强项？哪些卡了多久？下次该练什么？
- 心理场景: 这个用户的情绪基线在哪？最近触发因素是什么？哪种应对方式对她有效？
- 健身场景: 这个用户的力量水平？耐力？最近恢复状态？该练什么部位？

回答这些问题的能力就是 Process Agent 的核心智能。

8.2 两种实现模式

我们前面讨论过的两种路径，都是 User State Engine 的实现:

模式 A: Structured State (KG-based)

- 预先定义 schema（concept tree / pattern taxonomy）
- 每个 (user, schema_node) 维护结构化数据（mastery / FSRS / metadata）
- 适合**领域结构紧 + 用户密度高**的场景
- 例: PrepGo 的 AP 备考

模式 B: Inferred State (LLM-based)

- 不预定义 schema
- 需要时让 LLM 读 raw behavior history 推理用户状态
- 适合**领域结构松 + 用户密度低 / 长尾**的场景

- 例: 兴趣学习、罕见领域

模式 C: Hybrid（推荐的默认）

- Structured State 是底层，覆盖核心高密度领域
- Inferred State 是上层，处理结构未覆盖的情况
- 引擎对两者的访问统一接口，调用方不需要知道哪个在工作

8.3 通用 Schema 抽象

为了让引擎支持多种 vertical，schema 必须用通用术语定义:

代码块

```
CREATE TABLE schema_definitions (
    id TEXT PRIMARY KEY,                -- 'prepgo.apcalc.product_rule'
    vertical_id TEXT NOT NULL,
    schema_type TEXT NOT NULL,          -- 'topic' | 'pattern' | 'movement' | 'habit'

    name TEXT NOT NULL,
    description TEXT,
    parent_id TEXT,                     -- 树状层级
    prerequisite_ids TEXT[],

    metadata JSONB,                     -- vertical 特定信息（exam_weight, severity_level 等）
    embedding VECTOR(1536),

    is_official BOOLEAN DEFAULT TRUE,   -- false = 用户私有动态生成的节点
    map_version TEXT
);

CREATE TABLE user_state (
    user_id UUID,
    vertical_id TEXT,
    schema_node_id TEXT,

    -- 通用状态字段
    mastery_value FLOAT,                -- 0-1, "掌握度 / 健康度 / 熟练度"
    confidence FLOAT,                   -- 数据点是否充足
    last_updated_at TIMESTAMP,
    next_intervention_at TIMESTAMP,     -- 下次该 nudge 的时间

    -- 算法相关字段（FSRS 或类似）
    algorithm_state JSONB,              -- 算法内部状态

```

```
    PRIMARY KEY (user_id, vertical_id, schema_node_id)
);
```

注意 `mastery_value` 是抽象命名 —— 在学习是"掌握度"，在心理可能是"应对能力"，在健身是"该部位的训练状态"。语义随 vertical 变化，但**数据结构不变**。

8.4 状态更新的通用 Pipeline

User State 的更新流程在所有 vertical 都是同一个模式:

代码块

```
新 behavior event 进入
    ↓
Schema Matching: 这个事件触及哪些 schema 节点?
    (用 embedding + LLM 受限选择，前面 Concept Match pipeline 详细讲过)
    ↓
Outcome Analysis: 这个事件代表 positive / neutral / negative 信号?
    (有些事件是显式的（quiz 答对答错），有些需要 LLM 分析（写的反思的情绪倾向）)
    ↓
State Update: 用算法（FSRS 或自定义）更新对应 schema 节点的 user_state
    ↓
Trigger Evaluation: 这次更新是否触发其他动作?
    (如: mastery 跌破阈值 → 加入下次 nudge 候选)
```

这个 pipeline 在引擎层是通用的。每个 vertical 注入自己的 schema、outcome 分析逻辑、算法参数。

8.5 状态查询的通用 API

下游组件（Daily Ritual / Companion / Nudge）调用 State 的接口也应该统一:

代码块

```
// 通用 state query API
interface UserStateQuery {
  // 获取该用户在该 vertical 的状态摘要
  getStateSummary(userId, verticalId): StateSummary;

  // 获取最弱 N 个 schema 节点（用于 Daily Ritual 选内容）
  getWeakestNodes(userId, verticalId, n): SchemaNode[];

  // 获取到期需要"复习"的节点（用于 SRS 类调度）
  getDueNodes(userId, verticalId, asOf): SchemaNode[];

  // 让 LLM 用自然语言描述用户当前状态（用于 Companion 对话上下文）
```

```
    describeStateInPlainText(userId, verticalId): string;

    // 跨 schema 找相似的有掌握的节点（跨学科桥 / 跨领域类比）
    findRelatedMastered(sourceNodeId, userId): SchemaNode[];
  }
```

这套 API 让 Daily Ritual 不需要知道当前是 KG 还是 LLM 推理在工作，只关心"给我用户当前状态"。**抽象到位，未来引擎升级不需要修改 vertical 代码**。

---

9. 必备能力 3: Daily Ritual Framework（每日仪式框架）

9.1 它是什么

Daily Ritual Framework 负责**为每个用户每天生成一个"今日任务包"**。

这是 Process Agent 形成习惯回路的关键产品形态。它不是简单的"推荐引擎" —— 推荐引擎给你一个 feed 让你刷，daily ritual 给你一个完整的、有起止的、可以"完成"的任务包。

9.2 通用结构

不管什么 vertical，daily ritual 的结构都遵循同一个模式:

代码块

```
[今日任务包]
├── 卡片 1 (类型 A): 复习类
├── 卡片 2 (类型 B): 学习类
├── 卡片 3 (类型 C): 实践类
└── 卡片 4 (类型 D): 探索类

约束:
- 总时长在 10-20 分钟
- 卡片数量 3-5 张
- 完成 = 解锁 streak 和角色经验
- 内容个性化（基于 User State）
```

具体到不同 vertical:

卡片类型	学习	心理	健身
复习类	Recall（复习 flashcard）	回顾本周情绪	拉伸 / 恢复练习
学习类	Listen（音频讲解）	引导冥想 / 心理教育	学一个新动作
实践类	Try（quiz）	Today's reflection	主训练
探索类	Spark（拓展知识）	探索新练习	看示范视频

结构相同，内容因 vertical 而异。

9.3 卡片类型的抽象定义

为了让引擎支持任何 vertical，卡片类型要用通用术语定义:

代码块

```
interface RitualCardType {
  id: string;
  vertical_id: string;

  category: 'review' | 'learn' | 'practice' | 'explore';

  // 内容生成的逻辑
  contentSource: 'precomputed' | 'llm_generated' | 'hybrid';
  contentTemplate: string;        // 生成内容用的 prompt 模板

  // 时长和进度
  expectedDurationMinutes: number;

  // 状态影响
  contributesToStreak: boolean;
  generatesEvents: string[];       // 这张卡片完成后产生哪些 event_type
}
```

每个 vertical 上线时，定义自己的 4-5 个 card type，注册到引擎。引擎不需要硬编码任何特定 card type 的逻辑。

9.4 内容选择算法

Daily Ritual 的核心问题是 **"今天该选什么内容塞进卡片"**。这个算法在所有 vertical 都是同一套:

代码块

```python
def generate_daily_ritual(user_id, vertical_id, target_date):
    # 1. 拿用户当前状态摘要
    state = user_state_engine.get_summary(user_id, vertical_id)

    # 2. 拿 vertical 配置的 ritual schema
    ritual_schema = get_ritual_config(vertical_id)

    # 3. 对每张卡片决定内容
    cards = []
    for card_slot in ritual_schema.cards:
        if card_slot.category == 'review':
            content = pick_due_for_review(state)
        elif card_slot.category == 'practice':
            content = pick_weakest_targets(state)
        elif card_slot.category == 'learn':
            content = pick_learning_topic(state, recent_struggles)
        elif card_slot.category == 'explore':
            content = pick_exploration_content(state, user_interests)

        cards.append(generate_card(card_slot, content, state))

    # 4. 调整时长（如果总时长超预算，砍内容）
    cards = balance_duration(cards, target_minutes=12)

    # 5. 缓存
    save_today_ritual(user_id, target_date, cards)

    return cards
```

这个 pipeline 是领域无关的。每个 vertical 的差异在 `pick_*` 函数的具体实现里。

9.5 内容生成时机

Daily Ritual 内容应该**离线 batch 生成**，不要实时:

- 离线生成可以用更便宜的 batch API（半价）
- 离线生成可以用更长 context，质量更高
- 用户清晨打开时即时呈现，无延迟

这部分调度逻辑也是通用的，前面 Loop 文档里详细讲过（活跃度预测 + 三档生成策略）。

9.6 习惯回路的设计要点

Daily Ritual 不只是内容生成，更是**习惯回路的承载形式**。设计要点:

4. **完成是有限的**：今天就这么多，做完了就是做完了。不能"继续刷"。这是反 Duolingo / TikTok 模式。

5. **完成有明确反馈**：streak +1，角色经验 +1，能力可视化更新。让用户感受到"今天确实推进了"。
6. **不完成不惩罚**：用户偶尔错过一天不应该被 guilt-trip。这是反 Duolingo 模式。
7. **内容质量优于内容数量**：每张卡片都应该让用户感受到"这是为我准备的"。烂内容会破坏整个仪式的可信度。

---

10. 必备能力 4: Companion Engine（陪伴引擎）

10.1 它是什么

Companion Engine 负责**角色的人格、voice、和用户的互动**。

陪伴角色不是产品的装饰，是 Process Agent 区别于 Task Agent 的灵魂载体。它让产品从"工具"变成"伙伴"，让用户每天打开时面对的是一个"有人在这里"的感觉，而不是一个无名的 AI。

10.2 角色的多 Vertical 设计

一个有趣的设计问题：**当公司做多个 vertical 时，每个 vertical 应该有不同的角色，还是同一个角色？**

我们的判断:

短期（1-2 个 vertical）：每个 vertical 独立角色。这让品牌锐利，用户心智清晰。"PrepGo 的 Otter" 和"心理产品的某个角色"在用户脑子里是不同的东西。

长期（3+ 个 vertical 后）：考虑一个"角色家族"。统一的视觉语言（如水獭家族），但每个角色有自己的 vertical 特色和性格。这是 Pixar 模式 —— 不同电影不同角色，但有家族感。

10.3 角色的核心定义元素

每个角色都需要定义这些元素，这些元素一旦定义就保持稳定:

代码块

```
interface CompanionCharacter {
  id: string;
  vertical_id: string;

  // 视觉
  visual_assets: {
    base_form: ImageAsset;
    expressions: Map<string, ImageAsset>;
    contexts: Map<string, ImageAsset>;
  };

  // 人格
  personality: {
```

```
    core_traits: string[];        // ["warm", "witty", "occasionally cheeky"]
    voice_tone_default: string;
    forbidden_patterns: string[]; // ["no guilt-trip", "never cursing"]
  };

  // 多套 voice（按用户场景切换）
  voice_variants: Map<string, VoiceProfile>;

  // 行为约束
  behavioral_rules: {
    max_proactive_per_day: number;
    silence_during: string[];     // 什么场景下不出现
    growth_mechanics: GrowthRules;
  };
}
```

注意 `voice_variants` 这个字段。同一个角色可以有多套 voice —— 比如 Otter 在默认场景是"温暖机灵"，在考前模式是"冷静教练"，在兴趣学习是"好奇伙伴"。**人格不变，语调切换**。

10.4 角色行为的生成

角色每次"说话"都需要生成一段文本（push 通知文案、卡片完成后的反馈、Tutor 答完的评论等）。生成的通用流程：

代码块

```
def generate_companion_message(
    user_id,
    vertical_id,
    trigger_context,    # "user_completed_today" | "user_streak_at_risk" | ...
    character: CompanionCharacter
):
    # 1. 拿用户最新状态
    state = user_state_engine.describe_in_plain_text(user_id, vertical_id)

    # 2. 决定用哪种 voice variant
    voice = pick_voice_variant(state, trigger_context, character)

    # 3. 构造 prompt
    prompt = f"""
You are {character.name}, with these traits:
{character.personality.core_traits}.
Current voice mode: {voice.description}.
Forbidden: {character.personality.forbidden_patterns}.

User's current state: {state}
```

```
    Trigger: {trigger_context}

    Write a message (max {voice.max_length} chars) that fits the moment.
    """

        # 4. LLM 生成
        message = llm_call(prompt, model="haiku")  # 用便宜模型，文案不需要大模型

        # 5. 轻量过滤（确保符合 forbidden_patterns）
        return validate_and_clean(message, character)
```

这个 pipeline 也是领域无关的。每个 vertical 注入自己的 character 配置即可。

10.5 角色的成长机制

角色应该会**成长**，这是长期留存的关键钩子:

- 用户每天完成 daily ritual → 角色经验 +1
- 经验累积到一定程度 → 角色升级
- 升级解锁新的视觉元素（场景、装扮、表情）
- 长期累积形成"我的 Otter / 我的角色"的归属感

这部分机制在所有 vertical 都通用。每个 vertical 只需要定义自己的视觉解锁内容。

---

11. 必备能力 5: Proactive Nudge Engine（主动推送引擎）

11.1 它是什么

Proactive Nudge Engine 决定 **AI 何时主动出现 + 出现时说什么**。

这是 Process Agent 工程上最难、产品上最关键的部分。做对了让用户感受到"它真的懂我"，做错了变成 spam 让用户卸载。

11.2 推送的几种类型

按价值和频率排序:

类型 A: 仪式触发（每天 1 次）

- "你的今日 Loop 准备好了"
- 时机固定（用户设定的时间）
- 内容稳定

类型 B: 状态触发（不定期）

- "你的 streak 还有 6 小时就要断了"
- "看你这周三次问到 product rule，我安排了一个专项练习"
- 时机由用户状态变化触发

类型 C: 关键时刻触发（场景驱动）

- 考前 7 天："准备进入冲刺模式"
- 长时间不学习："最近还好吗？"
- 重大成就："你刚解锁了 Level 5"

类型 D: 情境触发（外部信号）

- 用户在 ChatGPT 反复问类似问题（如果有 MCP 集成）
- 用户连接的日历显示明天有考试
- 用户设备的运动数据显示昨天没动

类型 A 和 B 是 v1 必须做的。C 和 D 在产品成熟后逐步加。

11.3 推送决策的通用框架

每一个潜在推送都要经过四道关卡:

代码块

```
[候选推送事件触发]
    ↓
关卡 1: Value Check（有价值吗？）
  这个推送对用户当前状态真的有帮助吗?
  如果是泛泛的"来用我吧"，过滤掉。
    ↓
关卡 2: Timing Check（时机对吗？）
  现在推送会被打开吗？用户当下能接受吗?
  考虑：时间、地点、最近活跃模式
    ↓
关卡 3: Frequency Check（会打扰吗？）
  最近推送太多了吗？用户对类似推送的响应率?
  默认上限：每天 2 条
    ↓
关卡 4: Voice Generation（说什么？）
  让 Companion Engine 生成符合角色的文案
  内容必须具体（"你昨天的 product rule"）而非通用
    ↓
[推送发送]
    ↓
[追踪用户响应，反馈到 Frequency Check 的下次决策]
```

每一关都有清晰的拒绝理由。**宁可少推送，不要打扰**。

11.4 关键设计原则

原则 1: Specific > Generic

- 烂："Don't forget to study today!"
- 好："Maya, your product rule cards are due. 4 minutes."
- Process Agent 的所有推送都必须用 user state 让消息具体到无法被同质化产品复制。

原则 2: Helpful > Engaging

- 不为了 engagement 而推送。如果一个推送只是"让用户来开 app"，不该发。
- 每个推送都要回答"如果用户打开了，他的生活会变好吗？"

原则 3: Respect > Manipulate

- 拒绝 Duolingo 那种"猫头鹰要哭了"的内疚式推送
- 拒绝 dark pattern (虚假紧急感等)
- 用户的注意力是稀缺资源，要赢取而不是榨取

原则 4: Learn from Response

- 用户忽略一类推送 → 减少这类
- 用户响应一类推送 → 在该用户身上多用这类
- 个性化推送策略，不只是个性化推送内容

11.5 工程实现

代码块

```
class NudgeEngine:
    def evaluate_candidate(self, candidate: NudgeCandidate) -> NudgeDecision:
        # 关卡 1
        if not self.passes_value_check(candidate):
            return REJECT('low_value')

        # 关卡 2
        if not self.passes_timing_check(candidate):
            return DEFER(retry_at=better_time)

        # 关卡 3
        if not self.passes_frequency_check(candidate):
            return REJECT('too_many_recent')

        # 关卡 4
        message = companion_engine.generate(candidate)
```

```
        return APPROVE(message)

    def collect_candidates(self, user_id):
        # 聚合所有可能的推送来源
        candidates = []
        candidates.extend(self.daily_ritual_candidates(user_id))
        candidates.extend(self.state_change_candidates(user_id))
        candidates.extend(self.milestone_candidates(user_id))
        return candidates

    def decide_and_dispatch(self, user_id):
        candidates = self.collect_candidates(user_id)

        # 按预测价值排序
        candidates.sort(key=self.predict_value, reverse=True)

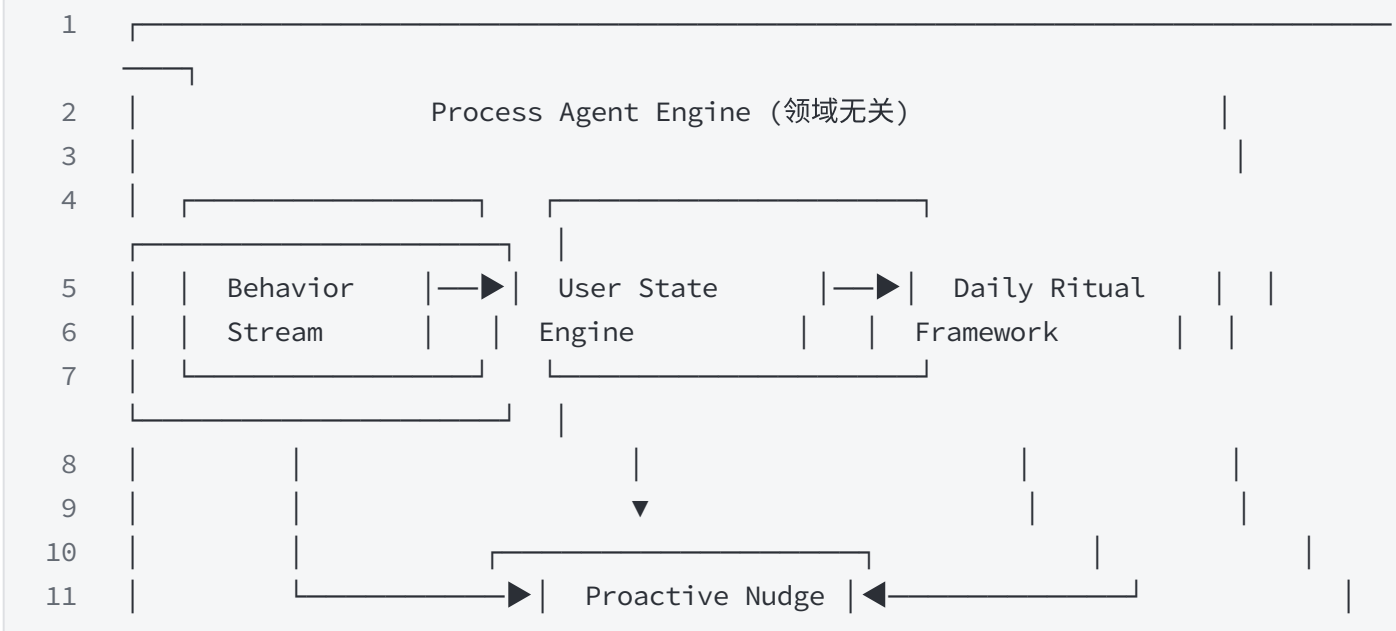
        # 取最有价值的，依次评估
        for candidate in candidates:
            decision = self.evaluate_candidate(candidate)
            if decision.is_approved:
                self.send(decision.message)
                if self.daily_quota_hit(user_id):
                    break
```

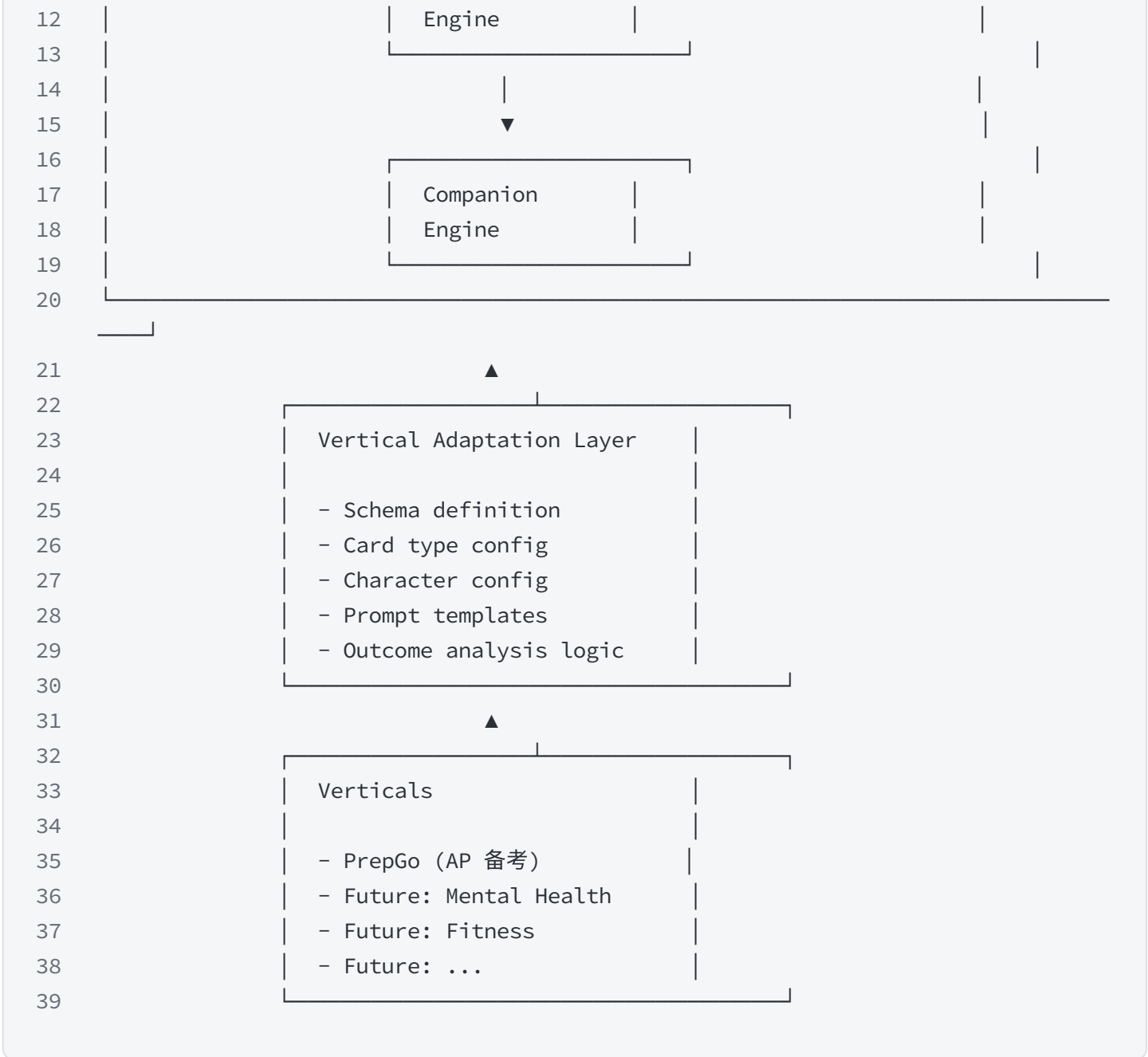
---

12. 架构总览与领域适配模式

12.1 完整架构总览

代码块





12.2 新 Vertical 上线的工作清单

要在引擎上加一个新 vertical（比如心理健康产品），需要做的工作:

领域内容工作（重点投入）

- [ ] 定义 schema（这个领域的"知识结构"是什么）
- [ ] 设计 card types（每日仪式包含哪几种内容）
- [ ] 设计 character（角色形象和人格）
- [ ] 撰写 prompt 模板（生成内容、生成对话）
- [ ] 撰写 outcome 分析逻辑（什么算 positive 信号）

适配工程工作（轻）

- [ ] 注册 schema 到引擎

- [ ] 注册 card types
- [ ] 注册 character
- [ ] 配置 vertical-specific 的事件类型
- [ ] 配置 nudge 候选生成逻辑

理想情况下：第一个 vertical（PrepGo）的内容工作量是 100，工程工作量是 100。第二个 vertical 的内容工作量是 100（因为内容本来就是新的），但工程工作量应该 < 30。**这就是引擎抽象的真正价值**。

12.3 跨 Vertical 数据共享

随着用户在多个 vertical 上活跃，可能有一天用户既学 AP 又用心理产品。这时是否共享数据?

默认不共享。每个 vertical 的数据独立。原因是隐私和心智清晰。

用户主动授权后可共享。比如用户允许心理产品看到 PrepGo 的考前焦虑数据。这给了用户控制权，也开启了"全维度个性化"的高级形态。

长期可能的形态：用户的"长期人类状态"成为一个独立的 layer，所有 vertical 都从中读写。但这是 5 年后的事，现在不需要为它过度设计。

---

13. 实施路径与里程碑

13.1 引擎构建的现实路径

不要试图先构建完整引擎再做产品。**应该在做 PrepGo 的过程中长出引擎**。

具体路径:

阶段 1（PrepGo 当前 ~ 6 个月）

- 优先做 PrepGo 的 KG + topic-level mastery（前面 PrepGo 文档里详细讲过）
- 实现时坚持 schema-replaceable 原则（用通用术语命名）
- Behavior Stream 直接用引擎抽象，不要 PrepGo-only 设计
- 不主动构建"引擎抽象层"，但每个组件设计上预留可扩展性

阶段 2（6-12 个月）

- PrepGo 的 KG / Daily Ritual / Companion 已经稳定
- 抽出"领域无关的部分"成为正式的引擎抽象
- 写出引擎的 API 文档
- 这一步基本是重构 + 文档化，不是新功能

阶段 3（12-18 个月）

- 选第二个 vertical 试水
- 用引擎构建（不是从零写）
- 验证"工程量 < 30%"的假设
- 暴露的引擎不足，回过头补强

阶段 4（18+ 个月）

- 真正的 multi-vertical Process Agent 平台
- 考虑跨 vertical 共享 layer

13.2 关键判断节点

每个阶段结束时要做一次明确的 go/no-go 判断:

阶段 1 末：PrepGo 的 KG 上线后，topic-level mastery 是否真的提升了 ARPU 和留存?

- 是 → 进入阶段 2
- 否 → 反思 KG 价值假设，可能要调整产品方向

阶段 2 末：引擎抽象出来后，是否真的让代码更清晰？是否暴露了第二个 vertical 应该选什么?

- 是 → 进入阶段 3
- 否 → 推迟扩展，深化 PrepGo

阶段 3 末：第二个 vertical 是否在工程量 < 30% 的预算内做出来？是否能复制 PrepGo 的 PMF 信号?

- 是 → 平台战略成立，进入阶段 4
- 否 → 引擎抽象不足，回到阶段 2 重做

不要跳过任何判断节点直接往后走。每一步都需要真实数据验证假设。

13.3 团队配置建议

阶段 1（PrepGo 深化期）

- 现有 PrepGo 团队基础上不需要新增
- 重点是内容运营 + KG 工程

阶段 2（引擎抽象期）

- 1 名架构师专门负责引擎抽象
- 现有产品工程师并行维护 PrepGo

阶段 3（第二 vertical 试水）

- 第二 vertical 需要 2-3 名工程 + 1-2 名内容 + 1 设计
- 引擎团队保持 1-2 人维护底层

阶段 4（平台化）

- 引擎团队扩到 3-5 人
- 每个 vertical 是独立 squad

13.4 关键风险与应对

风险 1: 引擎过度抽象 做引擎容易过度设计 —— 为想象中的 vertical 留了一堆"灵活性"，结果谁都用不上。 **应对**：抽象只在第二 vertical 的需求出现时才补强。在那之前不要因为"未来可能用得上"而提前优化。

风险 2: PrepGo 没赚到引擎红利 PrepGo 的产品决策被"引擎可移植性"束缚，最终 PrepGo 自己没做好。 **应对**：永远以 PrepGo 的产品成功为优先。引擎是副产品，不是主目标。

风险 3: LLM 成本曲线判断错误 我们押注 LLM 越来越便宜，让"长 context 推理"成为标配。如果这个曲线放缓，引擎的成本会失控。 **应对**：保留 KG 路径作为成本兜底。Hybrid 模式让两边都不押死。

风险 4: 第二 vertical 选错 扩展到错的 vertical 会浪费一年时间。 **应对**：第二 vertical 必须满足三个条件：用户有部分重合、领域结构能复用、引擎抽象能验证。心理健康可能不是好选择（合规复杂），相邻的考试领域（SAT / ACT / 大学课程）可能更稳。

附录：术语表

术语	定义
Task Agent	在单 session 内帮用户完成离散任务的 AI 产品（ChatGPT, Claude 等）
Process Agent	跨 session 陪伴用户长期过程的 AI 产品（本文档定义的新品类）
Vertical	Process Agent 服务的某个具体生活领域（学习、心理、健身等）
Behavior Stream	引擎的输入端，统一接收用户所有行为事件
User State Engine	维护用户长期状态模型的核心组件
Daily Ritual	用户每天的固定任务包，是习惯回路的承载形式
Companion	产品里的陪伴角色，有人格、voice、成长机制
Proactive Nudge	AI 主动出现的推送行为
Schema	某个 vertical 的"知识结构"定义
Schema-Replaceable	引擎设计原则：核心代码不绑定具体领域概念

文档结束

这份文档定义了我们公司的核心战略与技术愿景。它会随着我们对 Process Agent 这个新品类的理解深化而演进。重大修改需团队对齐。